Object Oriented Programming

cont.

CS 22B, Spring 2026

Module 05: Data Exploration

Presented by Jessica Huynh-Westfall, Ph.D.

02

AGENDA

- Inheritance

Class exercise

06

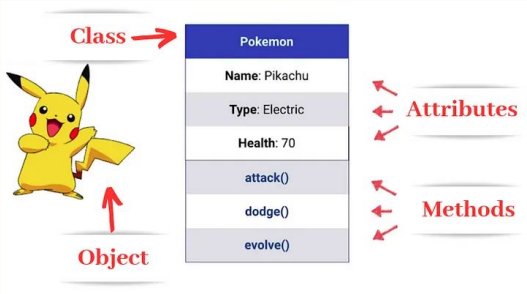
What are some reasons why you think you would choose to use Class over just functions and variables?

Talk to your partner and think about what is “real-world modeling” in programming.

Think : Pair : Share

Recap of OOP

04



Concept	Description	Example

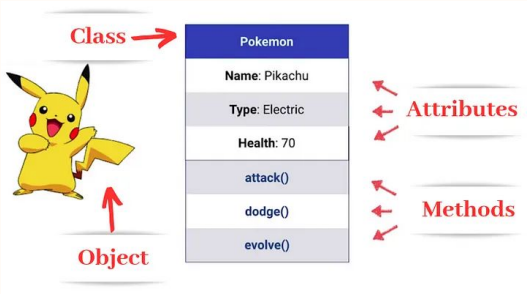
Class	A blueprint for objects	`class Pokemon:`
		`  def __init__(self, name, type):`
		`  self.name = name`
		`  self.type = type`

Object	Create an instance of a class	`pikachu = Pokemon("Pikachu", "Electric")`

Encapsulation	Keeping data safe inside objects.
	Private attributes
		`class Pokemon:`
		`  def __init__(self, name):`
		`  self.__name = name`

Recap of OOP

05



Concept	Description	Example

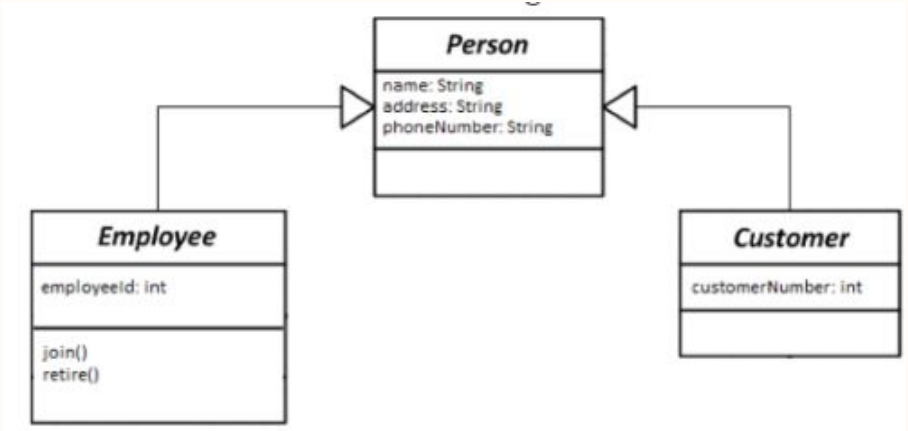
Abstraction	Methods that hide inner logic	`class Pokemon:`
		`  def __init__(self, name):`
		`  self.__name = name`
		`  def attack(self):`
		`  # complex logic hidden`
		`  print(f"{self.__name} uses Thunderbolt!")`

Inheritance	Reusing code from a parent class	`class Spring(Pikachu):`
Polymorphism	Same method name behaves differently	Overriding `attack()` in subclasses

Objects can **inherit** properties and behavio

Inheritance is inheriting common properties and behaviors from an existing class

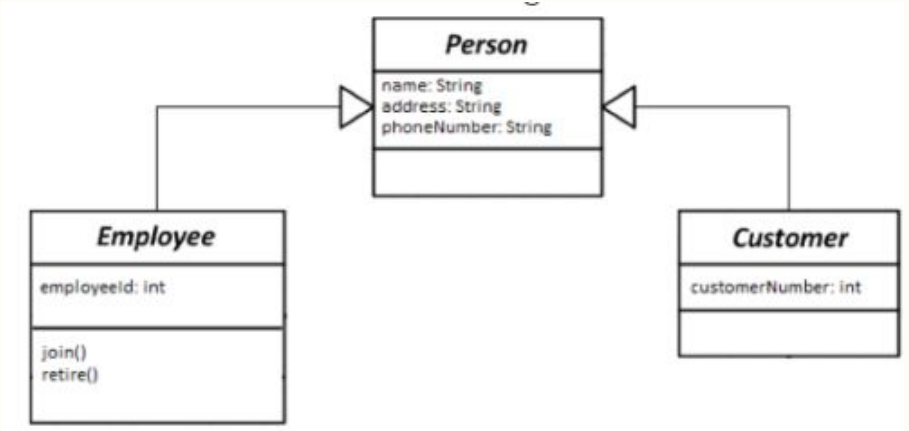
Allows us to extract the logic from one object (parent) to another object (child).



Parent (superclass) and Child (subclass)

Objects can inherit properties and behavior

Inheritance is inheriting common properties and behaviors from an existing class

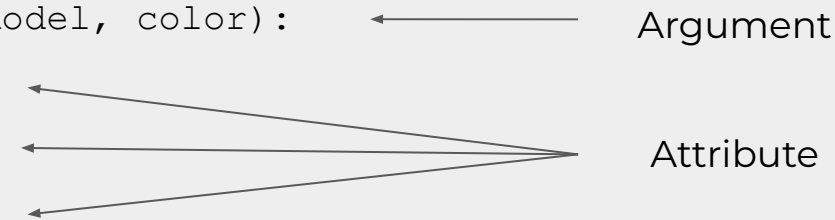


Allows us to extract the logic from one object (parent) to another object (child).

Parent class


```
### A Parent Class
class Car:
    '''Initializer method - set up the attributes for Car class'''
   def __init__(self, make, model, color):
       self.make = make
       self.model = model
       self.color = color

   def get_description(self):
       '''Instance method that returns a string description of the car.'''
       return "The {} {} is a {} color.".format(self.make, self.model, self.color)
```



Child class


```
### Child class
class GasCar(Car):
    def __init__(self, make, model, color, fuel_tank_size):
        super().__init__(make, model, color)
        self.fuel_tank_size = fuel_tank_size


    def get_fuel_tank_info(self):
        '''Instance method that returns a string description of the fuel
tank.'''
        return "The {} {} has a {} gallon fuel tank.".format(self.make,
self.model, self.fuel_tank_size)
```


super() gives you access to methods in superclass